Question Number 1:

```
namespace LINQ2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<int> numbers = [3, 18, 7, 42, 10, 5, 29, 14, 6, 100];

            // TODO: Write the query using Query Syntax
            Console.WriteLine("Query Syntax");
            var result1 = from n in numbers
                          where (n % 2 == 0) && (n > 10)
                          orderby n descending
                          select n;

            foreach (var num in result1)
            {
                Console.Write(num + " ");
            }
            Console.WriteLine();

            //TODO: Write the same query using Fluent Syntax
            Console.WriteLine("Fluent Syntax");
            var result2 = numbers.Where(n => (n % 2 == 0) && (n > 10))
                .OrderByDescending(n => n);

            foreach (var num in result2)
            {
                Console.Write(num + " ");
            }
          Console.ReadKey();
        }
    }
}
```

```
C:\Users\user\source\repos\L

Query Syntax
100 42 18 14
Fluent Syntax
100 42 18 14
```

Question Number 2:

Using the products list, demonstrate the difference between First / FirstOrDefault, Last / LastOrDefault, Single / SingleOrDefault, and ElementAt. Show what happens when no element matches — handle exceptions where needed.

Extension method	No element matches	handle exceptions where needed
First	InvalidOperationException	InvalidOperationException, if no elements match or sequence is empty
FirstOrDefault	return default value	No exception. only handle null value
Last	InvalidOperationException	InvalidOperationException, if no elements match or sequence is empty
LastOrDefault	return default value	No exception. only handle null value
Single	InvalidOperationException	InvalidOperationException, if sequence is empty or contains more than one element
SingleOrDefault	return default value	InvalidOperationException, if sequence contains more than one element
ElementAt	ArgumentOutOfRangeException	ArgumentOutOfRangeException, if index out of range

```
namespace LINQ2
{
    internal class Program
    {
        static void Main(string[] args)
        {

            List<Product> products =
            [
                new(1, "Laptop",1200m, "Electronics"),
                new(2, "Phone",800m, "Electronics"),
                new(3, "Desk",350m, "Furniture"),
                new(4, "Chair",150m, "Furniture"),
                new(5, "Headphones", 200m, "Electronics")

            ];

            // 1. Get the first Electronics product
            var result1 = products.First(p => p.Category == "Electronics");
            Console.WriteLine("First Electronics product");
            Console.WriteLine(result1);
            Console.WriteLine();
            // 2. Get the last product with Price > 1000 (use OrDefault — handle null)
            Console.WriteLine("Last product with Price > 1000");
            var result2 = products.LastOrDefault(p => p.Price > 1000);
            Console.WriteLine(result2 != null ? result2: "No product with Price > 1000");
            Console.WriteLine();
            // 3. Get the single Furniture item with Price > 300 (what if >1 match?)
            //what if >1 match? InvalidOperationException
            //Handle Exception
            try
            {
                var result3 = products.SingleOrDefault(p => p.Category == "Furniture" &&
p.Price > 300);
                Console.WriteLine("Single Furniture item with Price > 300");
                Console.WriteLine(result3);

            }
            catch (System.InvalidOperationException)
            {
                Console.WriteLine("More Than one item has Furniture item with Price > 300");
            }
```

```
            Console.WriteLine();
            // 4. Get the element at index 3
            var result4 = products.ElementAtOrDefault(3);
            Console.WriteLine("The element at index 3");
            Console.WriteLine(result4);
        }
    }
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public string Category { get; set; }
        public Product(int id, string name, decimal price, string category)
        {
            Id = id;
            Name = name;
            Price = price;
            Category = category;
        }
        public override string ToString()
        {
            return $"Id:{Id} | Name:{Name} | Price:{Price} | Category:{Category}";
        }
    }
}
```

```
Microsoft Visual Studio Debu

First Electronics product
Id:1 | Name:Laptop | Price:1200 | Category:Electronics

Last product with Price > 1000
Id:1 | Name:Laptop | Price:1200 | Category:Electronics

Single Furniture item with Price > 300
Id:3 | Name:Desk | Price:350 | Category:Furniture

The element at index 3
Id:4 | Name:Chair | Price:150 | Category:Furniture

C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (process 31140)
To automatically close the console when debugging stops, enable Tools->Optic
```

Question Number 3:

```
namespace LINQ2
{
    internal class Program
    {
        static void Main(string[] args)
        {

            List<Product> products =
            [
                new(1, "Laptop",1200m, "Electronics"),
                new(2, "Phone",800m, "Electronics"),
                new(3, "Desk",350m, "Furniture"),
                new(4, "Chair",150m, "Furniture"),
                new(5, "Headphones", 200m, "Electronics")

            ];

            //1.Are ALL products priced above 100 ?
            var r1 = products.All(p => p.Price > 100);
            Console.WriteLine($"Are ALL products priced above 100 ? {r1}");

            //2.Is THERE ANY product in the "Gaming" category ?
            var r2 = products.Any(p => p.Category == "Gaming");
            Console.WriteLine($"Is THERE ANY product in the \"Gaming\" category ? {r2}");
```

```
            //3.Does the collection CONTAIN a product named "Chair" ?
            //var r = products.Contains(new Product(4, "Chair", 150m, "Furniture")); → false
Product is a class (Reference Equality)
            var r3 = products.Any(p => p.Name == "Chair");
            Console.WriteLine($"Does the collection CONTAIN a product named \"Chair\" ?
{r3}");

            //4.Are ALL Electronics products priced above 500 ?
            var r4 = products.Where(p => p.Category == "Electronics").All(p => p.Price >
500);
            Console.WriteLine($"Are ALL Electronics products priced above 500 ? {r4}");

            //5.Is there ANY product cheaper than 200 ?
            var r5 = products.Any(p => p.Price < 200);
            Console.WriteLine($"Is there ANY product cheaper than 200 ? {r5}");

        }
    }
```

Debug | Any CPU | LINQ2

Microsoft Visual Studio Debu

```
Are ALL products priced above 100 ? True
Is THERE ANY product in the "Gaming" category ? False
Does the collection CONTAIN a product named "Chair" ? True
Are ALL Electronics products priced above 500 ? False
Is there ANY product cheaper than 200 ? True

C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (process 18696) exi
To automatically close the console when debugging stops, enable Tools->Options->
le when debugging stops.
Press any key to close this window . . .
```

Question Number 4:

```
namespace LINQ3
{
    internal class Program
    {
        static void Main(string[] args)
        {

            List<Product> products =
                [
```

```
            new(1, "Laptop",1200m, "Electronics"),
            new(2, "Phone",800m, "Electronics"),
            new(3, "Desk",350m, "Furniture"),
            new(4, "Chair",150m, "Furniture"),
            new(5, "Headphones", 200m, "Electronics"),
        ];

    //1.Convert to Array
    Console.WriteLine("----------ToArray----------");
    var Toarray= products.ToArray();
    foreach (var item in Toarray)
    {
        Console.WriteLine(item);
    }
    Console.WriteLine();

    //2.Convert to Dictionary keyed by Id
    Console.WriteLine("----------ToDictionary----------");
    var Todictionary = products.ToDictionary(p => p.Id);
    foreach (var item in Todictionary)
    {
        Console.WriteLine(item.Key);
        Console.WriteLine(item.Value);
    }
    Console.WriteLine();

    //3.Convert to HashSet of product Names
    Console.WriteLine("----------ToHashSet----------");
    var Tohashset = products.Select(p=>p.Name).ToHashSet();
    foreach (var item in Tohashset)
    {
        Console.WriteLine(item);
    }
    Console.WriteLine();

    //4.Convert to Lookup keyed by Category
    Console.WriteLine("----------ToLookup----------");
    var Tolookup=products.ToLookup(p => p.Category);
    foreach (var item in Tolookup)
    {
        Console.WriteLine(item.Key);
        foreach (var i in item)
        {
            Console.WriteLine(i);
```

```
                }
            }

            //✧ What exception does ToDictionary throw if keys are duplicated?
            //ArgumentException
            //✧ How does ToLookup handle duplicate keys differently?
            //it groups all elements that have the same key
        }
    }
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public string Category { get; set; }
        public Product(int id, string name, decimal price, string category)
        {
            Id = id;
            Name = name;
            Price = price;
            Category = category;
        }
        public override string ToString()
        {
            return $"Id:{Id} | Name:{Name} | Price:{Price} | Category:{Category}";
        }
    }
}
```

```
Microsoft Visual Studio Debu

----------ToArray----------
Id:1 | Name:Laptop | Price:1200 | Category:Electronics
Id:2 | Name:Phone | Price:800 | Category:Electronics
Id:3 | Name:Desk | Price:350 | Category:Furniture
Id:4 | Name:Chair | Price:150 | Category:Furniture
Id:5 | Name:Headphones | Price:200 | Category:Electronics

----------ToDictionary----------
1
Id:1 | Name:Laptop | Price:1200 | Category:Electronics
2
Id:2 | Name:Phone | Price:800 | Category:Electronics
3
Id:3 | Name:Desk | Price:350 | Category:Furniture
4
Id:4 | Name:Chair | Price:150 | Category:Furniture
5
Id:5 | Name:Headphones | Price:200 | Category:Electronics

----------ToHashSet----------
Laptop
Phone
Desk
Chair
Headphones

----------ToLookup----------
Electronics
Id:1 | Name:Laptop | Price:1200 | Category:Electronics
Id:2 | Name:Phone | Price:800 | Category:Electronics
Id:5 | Name:Headphones | Price:200 | Category:Electronics
Furniture
Id:3 | Name:Desk | Price:350 | Category:Furniture
Id:4 | Name:Chair | Price:150 | Category:Furniture

C:\Users\user\source\repos\LINQ3\bin\Debug\net10.0\LINQ3.exe (process 31724) exited with cod
Press any key to close this window . . .
```

Question Number 5:

```
namespace LINQ2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<string> orders = ["ORD-001", "ORD-002", "ORD-003",
                                   "ORD-004", "ORD-005", "ORD-006", "ORD-007"];
```

```
            // 1. Get Page 1 (items 1–3)
            var r1 = Pagination.Paginate(orders,1, 3);
            foreach (var item in r1)
            {
                Console.Write(item+" ");
            }
            Console.WriteLine();
            // 2. Get Page 2 (items 4–6)  ← use Skip + Take
            var r2 = Pagination.Paginate(orders, 2, 3);
            foreach (var item in r2)
            {
                Console.Write(item+" ");
            }
            Console.WriteLine();
            // 3. Get the last 2 orders using TakeLast
            var r3 = orders.TakeLast(2);
            Console.WriteLine("Last 2 orders");
            foreach (var item in r3)
            {
                Console.Write(item + " ");
            }
            Console.WriteLine();
            // 4. Drop the first and last order using Skip + SkipLast
            var r4= orders.Skip(1).SkipLast(1);
            Console.WriteLine("Drop the first and last order");
            foreach (var item in r4)
            {
                Console.Write(item + " ");
            }
            Console.WriteLine();

        }
    }

}
using System;
using System.Collections.Generic;
using System.Text;
namespace LINQ2
{
    public static class Pagination
    {
        public static IEnumerable<T> Paginate<T>(this IEnumerable<T> source, int pageNumber =
1, int pageSize = 3)
```

```
        {
            Console.WriteLine($"Page number={pageNumber}");
            if (pageNumber <= 0) { pageNumber = 1; }
            if (pageSize <= 0) { pageSize = 3; }
            var data = source.Skip((pageNumber - 1) * pageSize).Take(pageSize).ToList();
            return data;
        }
    }
}
```

```
Microsoft Visual Studio Debu

Page number=1
ORD-001 ORD-002 ORD-003
Page number=2
ORD-004 ORD-005 ORD-006
Last 2 orders
ORD-006 ORD-007
Drop the first and last order
ORD-002 ORD-003 ORD-004 ORD-005 ORD-006

C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (pro
To automatically close the console when debugging stops, enable T
le when debugging stops.
Press any key to close this window . . .
```

Question Number 6:

```
namespace LINQ2
{
    internal class Program
    {
        public record Employee(string Name, string Department, decimal Salary);
        static void Main(string[] args)
        {
            List<Employee> employees =
```

```
            [
                new("Ali","Engineering", 9000m),
                new("Sara", "Engineering", 8500m),
                new("Mona", "HR",6200m),
                new("Yara", "Marketing", 7000m),
                new("Karim", "Marketing", 7500m),
                new("Nada", "Engineering", 9500m),

            ];
            // 1. Project to anonymous type: { FullName = Name.ToUpper(), Salary }
            var r1 = employees.Select(e => new
            {
                FullName = e.Name.ToUpper(),
                Salary = e.Salary
            });
            foreach (var item in r1)
            {
                Console.WriteLine($"FullName = {item.FullName} , Salary = {item.Salary}");
            }
            Console.WriteLine("------------------------");
            // 2. Project to a formatted string: "Ali works in Engineering — EGP 9,000"
            //using DTOs
            var r2 = employees.Select(e => new EmployeeDTO
            {
                Name = e.Name,
                Department = e.Department,
                Salary = e.Salary
            });
            foreach (var item in r2)
            {
                Console.WriteLine($"{item.Name} works in {item.Department} - EGP
{item.Salary}");
            }

            Console.WriteLine("--------------------------");
            // 3. Sort by Salary descending, then use indexed Select to add Rank:
            var r3 = employees
            .OrderByDescending(e => e.Salary)
            .Select((e, index) => new
            {
                Rank = index + 1,
                Name = e.Name,
                Salary = e.Salary
            });
```

```
            foreach (var item in r3)
            {
                Console.WriteLine($"Rank = {item.Rank} , Name = {item.Name} , Salary =
{item.Salary}");
            }
            Console.WriteLine("--------------------------");

            // Project each employee to include a "SeniorityLevel" property:
            var r4 = employees.Select(e => new
            {
                Name = e.Name,
                Salary = e.Salary,
                SeniorityLevel = e.Salary >= 9000 ? "Senior" :
                e.Salary >= 7000 ? "Mid" : "Junior"
            });
            foreach (var item in r4)
            {
                Console.WriteLine($"Name = {item.Name} , Salary = {item.Salary} , Seniority
Level = {item.SeniorityLevel}");
            }
            Console.WriteLine();
        }
    }
        public class EmployeeDTO
        {
            public string Name { get; set; }
            public string Department { get; set; }
            public decimal Salary { get; set; }

        }

    }
```

```
Microsoft Visual Studio Debu
FullName = ALI , Salary = 9000
FullName = SARA , Salary = 8500
FullName = MONA , Salary = 6200
FullName = YARA , Salary = 7000
FullName = KARIM , Salary = 7500
FullName = NADA , Salary = 9500
------------------------
Ali works in Engineering - EGP 9000
Sara works in Engineering - EGP 8500
Mona works in HR - EGP 6200
Yara works in Marketing - EGP 7000
Karim works in Marketing - EGP 7500
Nada works in Engineering - EGP 9500
--------------------------
Rank = 1 , Name = Nada , Salary = 9500
Rank = 2 , Name = Ali , Salary = 9000
Rank = 3 , Name = Sara , Salary = 8500
Rank = 4 , Name = Karim , Salary = 7500
Rank = 5 , Name = Yara , Salary = 7000
Rank = 6 , Name = Mona , Salary = 6200
---------------------------
Name = Ali , Salary = 9000 , Seniority Level = Senior
Name = Sara , Salary = 8500 , Seniority Level = Mid
Name = Mona , Salary = 6200 , Seniority Level = Junior
Name = Yara , Salary = 7000 , Seniority Level = Mid
Name = Karim , Salary = 7500 , Seniority Level = Mid
Name = Nada , Salary = 9500 , Seniority Level = Senior


C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (process 23512) exited with
Press any key to close this window . . .
```

Question Number 7:

```
namespace LINQ3
```

```
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<int> scores = [88, 92, 75, 60, 55, 80, 91, 45];
            // 1. TakeWhile score >= 70 → expected: [88, 92, 75]
            var r1 = scores.TakeWhile(s => s >= 70);
            foreach (var i in r1)
            {
                Console.Write(i+" ");
            }
            Console.WriteLine();
            // 2. SkipWhile score >= 70 → expected: [60, 55, 80, 91, 45]
            var r2 = scores.SkipWhile(s => s >= 70);
            foreach (var i in r2)
            {
                Console.Write(i + " ");
            }
            Console.WriteLine();

// 3. What is the difference between this and using Where? Explain in a comment.
// TakeWhile: takes elements from the beginning while the condition is true
// Once the condition becomes false it stops even if there are other elements matches the
condition

// SkipWhile: skips elements from the beginning while the condition is true
// Once the condition becomes false it returns the remaining elements without checking again

// Where: checks all elements in the collection and returns every element that matches the
condition
        }
    }
}
```

```
Microsoft Visual Studio Debu

88 92 75
60 55 80 91 45

C:\Users\user\source\repos\LINQ3\bin\Debug
```

Question Number 8:

```
namespace LINQ2
{
    internal class Program
    {
        public record Employee(string Name, string Department, decimal Salary);

        static void Main(string[] args)
        {

            List<Employee> employees =
            [
                new("Ali","Engineering", 9000m),
                new("Sara", "Engineering", 8500m),
                new("Mona", "HR",6200m),
                new("Yara", "Marketing", 7000m),
                new("Karim", "Marketing", 7500m),
                new("Nada", "Engineering", 9500m),

            ];
            // 1. Group by Department, print: "Engineering → Count: 3, Avg: 9000"
            var result1 = employees.GroupBy(e => e.Department)
                .Select(e => new
                {
                    Department=e.Key,
                    Count=e.Count(),
                    Avg=e.Average(a=>a.Salary)
                });
            foreach (var i in result1)
            {
                Console.WriteLine($"{i.Department} | Count: {i.Count} | Avg: {i.Avg}");
            }
            Console.WriteLine();
            // 2. Find the department with the highest total salary budget
            var result2 = employees.GroupBy(e => e.Department)
                .Select(e => new
                {
                    Department = e.Key,
                    Totalsalary = e.Sum(a => a.Salary)
                }).MaxBy(e=>e.Totalsalary);
            Console.WriteLine(result2);
            Console.WriteLine();

            // 3. List employees in each group ordered by Salary descending
```

```
            var result3 = employees.GroupBy(e => e.Department)
                .Select(e => new
                {
                    Department = e.Key,
                    Employees = e.OrderByDescending(e => e.Salary)
                });
            foreach (var i in result3)
            {
                Console.WriteLine($"{i.Department} ");
                foreach (var emp in i.Employees)
                {
                    Console.WriteLine($"{emp.Name} | {emp.Salary}");
                }
                Console.WriteLine();
            }
            Console.WriteLine();
        }
    }
}
```

```
Microsoft Visual Studio Debu

Engineering | Count: 3 | Avg: 9000
HR | Count: 1 | Avg: 6200
Marketing | Count: 2 | Avg: 7250

{ Department = Engineering, Totalsalary = 27000 }

Engineering
Nada | 9500
Ali | 9000
Sara | 8500

HR
Mona | 6200

Marketing
Karim | 7500
Yara | 7000



C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (process 449
```

Question Number 9:

```
namespace LINQ3
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<int> nums = [1, 2, 3, 4, 5];
            var query = nums.Where(n => n > 2);
            var query1 = nums.Where(n => n > 2).ToList();

            nums.Add(10);
            foreach (var n in query)
                Console.Write(n + " ");
            Console.WriteLine();
            foreach (var n in query1)
                Console.Write(n + " ");

        }
    }
}
```

```
Microsoft Visual Studio Debu
3 4 5 10
3 4 5
C:\Users\user\source\repos\LINQ3\bin\Debug\net10.
```

Q: What is printed? Why?

3 4 5 10, because 'where' uses deferred execution the query is not executed now, it will be executed when iterating in a loop.

Q: How would using .ToList() right after .Where(...) change the result?

ToList() uses immediate execution returns the result immediately so that after adding 10 the result does not change 3 4 5

Q: Name 3 LINQ operators that trigger immediate execution.

First(), Last(), ToArray(), ToDictionary()

Question Number 10:

```
namespace LINQ3
{
    internal class Program
    {
        static void Main(string[] args)
        {
            List<string> words = ["apple", "fig", "banana", "kiwi",
                                  "grape", "mango", "pear", "plum"];
            // 1. Filter words longer than 4 characters
            var r1 = words.Where(w => w.Length > 4);
            foreach (var w in r1)
            {
                Console.Write(w+" ");
            }
            Console.WriteLine("\n");
            // 2. Filter words at even indexes (0, 2, 4, 6...) using (item, index) overload
            var r2 = words.Where((w, Index) => Index % 2 == 0);
            foreach (var w in r2)
            {
                Console.Write(w + " ");
            }
            Console.WriteLine("\n");
            // 3. Filter words that are BOTH longer than 4 chars AND at an even index
            var r3 = words.Where((w, Index) => Index % 2 == 0 && w.Length > 4);
            foreach (var w in r3)
            {
                Console.Write(w + " ");
            }
            Console.WriteLine();
            // 4. What is the index of "mango" in the filtered result from step 1?
            //(apple banana grape mango)-> 3
        }
    }
}
```

```
Microsoft Visual Studio Debu
apple banana grape mango

apple banana grape pear

apple banana grape
```

Question Number 11:

```
namespace LINQ3
{
    internal class Program
    {
        record Course(string Title, List<string> Students);
        static void Main(string[] args)
        {
            List<Course> courses =
            [
                new("C# Basics", ["Ali", "Sara", "Omar"]),
                new("LINQ Mastery", ["Sara", "Mona", "Ali"]),
                new("ASP.NET Core", ["Yara", "Omar", "Karim"]),
            ];

            // 1. Flatten to a single list of ALL student names (with duplicates)
            Console.WriteLine("List of ALL student names");
            var r1=courses.SelectMany(c => c.Students);
            foreach (var c in r1)
            {
                Console.WriteLine(c);
            }
            Console.WriteLine();

            // 2. Get a distinct list of all student names
            Console.WriteLine("Distinct list of all student names");
            var r2 = courses.SelectMany(c => c.Students).Distinct();
            foreach (var c in r2)
            {
                Console.WriteLine(c);
            }
            Console.WriteLine();

            // 3. Find students who appear in MORE THAN ONE course
         //return names->divide them into groups->if name repeated more than once->select it
```

```
            Console.WriteLine("Students who appear in MORE THAN ONE course");
            var r3 = courses
                .SelectMany(c => c.Students)
                .GroupBy(c => c)
                .Where(c=>c.Count()>1)
                .Select(c=>c);
            foreach (var c in r3)
            {
                Console.WriteLine(c.Key);
            }
            Console.WriteLine();

            // 4. Use SelectMany with result selector to get (CourseName, StudentName) pairs
            Console.WriteLine("(CourseName, StudentName)");
            var r4 = courses.SelectMany(
                c => c.Students,
                (t, s) => new
                {
                    CourseName = t.Title,
                    StudentName = s
                });
            foreach (var c in r4)
            {
                Console.WriteLine($"({c.CourseName} , {c.StudentName})");
            }
        }
    }
}
```

```
Microsoft Visual Studio Debu

List of ALL student names
Ali
Sara
Omar
Sara
Mona
Ali
Yara
Omar
Karim

Distinct list of all student names
Ali
Sara
Omar
Mona
Yara
Karim

Students who appear in MORE THAN ONE course
Ali
Sara
Omar

(CourseName, StudentName)
(C# Basics , Ali)
(C# Basics , Sara)
(C# Basics , Omar)
(LINQ Mastery , Sara)
(LINQ Mastery , Mona)
(LINQ Mastery , Ali)
(ASP.NET Core , Yara)
(ASP.NET Core , Omar)
(ASP.NET Core , Karim)

C:\Users\user\source\repos\LINQ3\bin\Debug\net10.0\LINQ3.exe (process 33388) exited with c
Press any key to close this window . . .
```

Question Number 12:

```
namespace LINQ2
{
    internal class Program
    {
        public record Employee(string Name, string Department, decimal Salary);
        record Course(string Title, List<string> Students);

        static void Main(string[] args)
        {

            List<Employee> employees =
            [
                new("Ali","Engineering", 9000m),
                new("Sara", "Engineering", 8500m),
                new("Mona", "HR",6200m),
                new("Yara", "Marketing", 7000m),
                new("Karim", "Marketing", 7500m),
                new("Nada", "Engineering", 9500m),

            ];

            List<Course> courses =
            [
                new("C# Basics", ["Ali", "Sara", "Omar"]),
                new("LINQ Mastery", ["Sara", "Mona", "Ali"]),
                new("ASP.NET Core", ["Yara", "Omar", "Karim"]),
            ];
            //1. From employees: get the TOP 2 highest-paid employees per department.
            //→ Use GroupBy +OrderByDescending + Take(2) inside each group
            //→ Flatten the result with SelectMany
            var r1 = employees
            .GroupBy(e => e.Department)
            .Select(e => e.OrderByDescending(e => e.Salary))
            .SelectMany(e=>e.Take(2));//Deferred Execution

            foreach (var e in r1)
            {
                Console.WriteLine(e);
            }
            Console.WriteLine();
            // From courses: build a Dictionary<string, int> of
```

```
            //{ CourseName → StudentCount }, but only for courses with > 2 students.
            Dictionary<string, int> dict = new Dictionary<string, int>();

            dict = courses.Where(c => c.Students.Count > 2)
                .ToDictionary(c => c.Title,c => c.Students.Count);//Immediate Execution

            foreach (var e in dict)
            {
                Console.WriteLine(e.Key + " " + e.Value);
            }
            Console.WriteLine();
            // 3.Check:
            // ✧ Does ANY employee in Engineering earn less than 8000 ?
            // ✧ Do ALL HR employees earn more than 5500 ?
            var r2 = employees.Any(e => e.Department == "Engineering" && e.Salary <
8000m);//Immediate Execution
            Console.WriteLine(r2);
            var r3 = employees.Where(e=>e.Department== "HR").All(e => e.Salary >
5500m);//Immediate Execution
            Console.WriteLine(r3);
            ////Project the top-2-per-dept result into:
            //{ Rank, Name, Department, Salary, SeniorityLevel }
            //where Rank resets per department(use indexed Select per group).
            var r4 = employees.Select((e,index) => new

            {
                Rank= index+1,
                Name = e.Name,
                Department = e.Department,
                Salary = e.Salary,
                SeniorityLevel = e.Salary >= 9000 ? "Senior" :
                e.Salary >= 7000 ? "Mid" : "Junior"
            });//Deferred Execution
            foreach (var item in r4)
            {
                Console.WriteLine($"Rank = {item.Rank} , Name = {item.Name} , Department =
{item.Department} , Salary = {item.Salary} , Seniority Level = {item.SeniorityLevel}");
            }
            Console.WriteLine();

        }
    }
```

```
}
```

```
Microsoft Visual Studio Debu

Employee { Name = Nada, Department = Engineering, Salary = 9500 }
Employee { Name = Ali, Department = Engineering, Salary = 9000 }
Employee { Name = Mona, Department = HR, Salary = 6200 }
Employee { Name = Karim, Department = Marketing, Salary = 7500 }
Employee { Name = Yara, Department = Marketing, Salary = 7000 }

C# Basics 3
LINQ Mastery 3
ASP.NET Core 3

False
True
Rank = 1 , Name = Ali , Department = Engineering , Salary = 9000 , Seniority Level = Senior
Rank = 2 , Name = Sara , Department = Engineering , Salary = 8500 , Seniority Level = Mid
Rank = 3 , Name = Mona , Department = HR , Salary = 6200 , Seniority Level = Junior
Rank = 4 , Name = Yara , Department = Marketing , Salary = 7000 , Seniority Level = Mid
Rank = 5 , Name = Karim , Department = Marketing , Salary = 7500 , Seniority Level = Mid
Rank = 6 , Name = Nada , Department = Engineering , Salary = 9500 , Seniority Level = Senior


C:\Users\user\source\repos\LINQ2\bin\Debug\net10.0\LINQ2.exe (process 26848) exited with code 0 (0x0).
Press any key to close this window . . .
```